

Penetration Test

oltre il tool, oltre il sito, oltre la teoria

Sintesi finale del corso: digitale, fisico, umano, processo e reporting
Caso guida: UmbraMarket • Dallo scope al report finale

Perché questa presentazione

Chiudere il corso con un quadro unico e professionale

- Riassumere che cos'è davvero un penetration test e come si svolge end-to-end.
- Mostrare i pattern principali: scope, recon, mapping, baseline, test controllato, evidenza, report.
- Unire il mondo digitale con quello fisico, umano, radio e di processo.
- Rimettere al centro un'idea chiave: i tool aiutano, ma senza comprensione del sistema valgono poco.
- Lasciare ai ragazzi un modello di lavoro che possano riusare in contesti reali.

Che cos'è un penetration test?

Non un insieme di exploit, ma un lavoro di verifica dei confini di sicurezza

Un penetration test verifica, con autorizzazione ed evidenza, se i confini di sicurezza di un'organizzazione esistono davvero e se reggono nelle condizioni reali.

- Può riguardare software, reti, API, identità, processi, persone, luoghi fisici e superfici radio.
- Non termina con "ho trovato un bug": termina con evidenza, impatto, remediation e retest.
- Conta ciò che viene dimostrato, non ciò che è soltanto ipotizzato.

Vulnerability Assessment ≠ Penetration Test

Sono complementari, non equivalenti

Vulnerability Assessment (VA)

Ampiezza: cerca molti problemi potenziali in poco tempo.

Lavora bene con scanner, versioni, configurazioni, CVE, pattern noti.

Risponde alla domanda: "dove potrebbe esserci qualcosa da verificare?"

Produce candidate finding, non necessariamente vulnerabilità confermate.

Penetration Test (PT)

Profondità: valida manualmente le ipotesi nel contesto reale.

Si concentra su business rule, trust boundary, autenticazione, autorizzazione, flussi e impatto.

Risponde alla domanda: "questo controllo fallisce davvero? cosa ottengo?"

Produce evidenza, exploitability dimostrata, remediation e piano di retest.

Il flusso completo del penetration test

Il pattern che vogliamo lasciare ai ragazzi



- Ogni passaggio prepara il successivo: saltarne uno indebolisce il lavoro.
- Il report finale non è un allegato burocratico: è il punto in cui l'evidenza diventa decisione.

Si comincia dallo scope

Prima di attaccare si definisce cosa è autorizzato

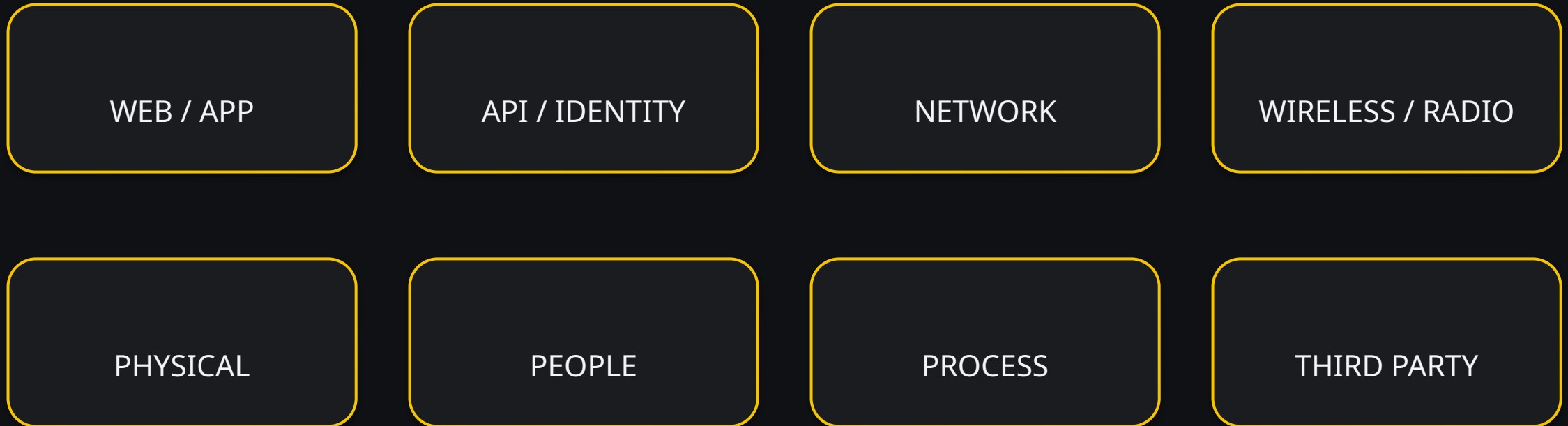
- Asset in scope e out of scope.
- Account e identità autorizzate.
- Regole: cosa posso fare, cosa non posso fare, quando devo fermarmi.
- Orizzonte del test: black box, grey box, white box.
- Vincoli: niente DoS, niente mass extraction, niente persistence, niente test fuori perimetro.

Perché è decisivo

Lo scope protegge il cliente e protegge il tester.
Decide il significato tecnico e legale del test.
Permette di leggere correttamente i risultati: un “non testato” non è un “sicuro”.
Impedisce di confondere aggressività con professionalità.

La superficie di attacco è più grande del “sito web”

Il corso si chiude allargando il modello



- Il pentest verifica boundary di sicurezza digitali, fisici, umani e organizzativi.
- Il mezzo cambia, ma il ragionamento resta lo stesso.

Reconnaissance: capire, non ancora “bucare”

La qualità dell'osservazione determina la qualità del test

- Quali servizi esistono? quali pagine? quali endpoint? quali identità? quali flussi?
- Quali tecnologie e quali naming convention emergono?
- Quali parametri, object ID, ruoli, percorsi, file pubblici, JavaScript, headers e cookie sono visibili?
- Il risultato della recon non è una vulnerabilità: è una mappa del sistema.

Più comprendi il sistema, più il test smette di essere casuale e diventa mirato.

Baseline e modifica controllata

Uno dei pattern più importanti di tutto il corso

Prima osserva il caso legittimo

Alice legge il proprio ordine.
Bob legge il proprio ordine.
Una ricerca normale restituisce il risultato atteso.
Un greeting normale mostra solo testo.

Poi cambia una sola variabile

1001 → 1002
/orders/{id} → /admin/stats
q=Umbra → input SQL
name=Alice → markup
controllato

E confronta

Expected vs Observed
Stessa sessione, stesso flusso,
una differenza isolata.
Così l'evidenza è molto più forte.

Un buon test non cambia "tutto": cambia il minimo necessario per dimostrare il failure del controllo.

Evidenza, capability gained e stop condition

Il pentester professionale sa anche quando fermarsi

- Una request o un'azione produce un comportamento inatteso: quella è osservazione.
- Se il comportamento rompe una regola di sicurezza e lo si ripete in modo controllato: quella è validazione.
- La domanda successiva è: "quale capability ho ottenuto davvero?"
- Dopo la prima prova sufficiente, ci si può fermare: non serve massimizzare il danno per dimostrare il finding.

Esempi di capability

Accesso cross-user a un ordine

Accesso customer a una funzione admin

Query manipulation

~~JavaScript eseguito nel browser~~

Da tenere separato

VALIDATED

ASSUMED

NOT TESTED

OUT OF SCOPE

STOPPED

I tool non sono tutto

Sono utili, ma senza contesto non bastano

Un tool vede ciò che è stato programmato per cercare. Il pentester deve capire ciò che il sistema sta facendo e quale regola di sicurezza dovrebbe far rispettare.

- Uno scanner può vedere una porta, una versione, un header, un endpoint, un cookie, una firma.
- Non capisce automaticamente la business rule: “Alice non deve leggere l’ordine di Bob”.
- Per bypassare un controllo devi prima capire come dovrebbe funzionare e dove quel controllo vive.

I tool più noti che avete visto (o sentito) nel corso

Ricordarli sì, idolatrarli no

Nmap

Servizi, porte, enumerazione iniziale.

Nessus / Greenbone

Scanner VA: CVE, configurazioni, pattern noti.

Burp Suite / OWASP ZAP

Proxy, repeater, osservazione e manipolazione HTTP.

Nikto / Nuclei

Pattern web e template di controllo automatizzati.

sqlmap

Automazione su superfici di injection già individuate.

curl / browser / DevTools

Spesso i tool più semplici sono quelli decisivi.

Il tool aiuta a testare un'ipotesi. Non sostituisce l'ipotesi.

Prima del tool: capire il sistema

L'ordine mentale corretto del lavoro



- Come funziona l'autenticazione? Come viene determinato il ruolo?
- Quali oggetti esistono e chi dovrebbe potervi accedere?
- Quali funzioni sono amministrative? quali sono pubbliche? quali fidano input esterni?
- Quali assunzioni sta facendo il sistema e quale di quelle vale la pena verificare?

Il vero bersaglio sono i trust boundary e le business rule

Ogni feature nasconde una domanda di sicurezza

Feature

“Visualizza ordine”
“Ricerca prodotti”
“Admin stats”
“Greeting / welcome”

Assunzione di sicurezza

Puoi vedere solo i tuoi ordini
L'input è solo dato, non query
Serve un ruolo amministrativo
L'input viene mostrato come testo sicuro

Domanda del tester

Il server verifica ownership?
La query è parametrizzata?
Il ruolo è controllato sul server?
C'è output encoding contestuale?

Caso guida: UmbraMarket

Dallo scope locale al report finale — il laboratorio come mini VAPT end-to-end

UmбраMarket — ambiente di laboratorio

Tre target principali e un percorso didattico chiaro

Target

127.0.0.1:5005 — applicazione principale

127.0.0.1:8080 — admin staging

127.0.0.1:9090 — file service

Account

alice / Alice123!

bob / Bob123!

admin / Admin123!

Obiettivo didattico

Fare recon

Formulare ipotesi

Validare finding reali

Scrivere remediation e retest

Produrre un report professionale

I cinque finding principali del caso

Non come elenco sterile, ma come esercizio di reasoning

F-01 — IDOR / BOLA sugli ordini

Severity: High

F-02 — Broken Function Level Authorization

Severity: Medium

F-03 — SQL Injection nella search

Severity: High

F-04 — Reflected XSS nel greeting

Severity: Medium

F-05 — Information Disclosure sullo staging

Severity: Low

F-01 — Customer può leggere l'ordine di un altro customer

Il classico boundary orizzontale che fallisce

Pattern

Alice legge il proprio ordine (baseline).
Bob legge il proprio ordine (baseline).
Si mantiene la sessione Alice e si cambia solo l'ID dell'ordine.
Il server restituisce l'ordine di Bob: ownership check mancante.

Cosa dimostra davvero

Accesso cross-user a un singolo oggetto.
Movimento orizzontale applicativo.
Non dimostra: dump completo, scrittura, takeover, shell.

La vulnerabilità non è "l'ID è numerico". La vulnerabilità è che il server prende una decisione di authorization errata.

F-02 — Customer può invocare una funzione amministrativa

Differenza definitiva fra authentication e authorization

Catena minima

Sessione customer valida
Richiesta verso /api/admin/stats
Il server controlla che l'utente sia autenticato
Ma non controlla il ruolo/permesso richiesto
Risposta 200 con statistiche amministrative

Messaggio chiave

Authentication = chi sei
Authorization = cosa puoi fare
Essere loggati non significa essere autorizzati

Remediation

Role / permission check server-side
Deny by default
Test negativi customer → admin
Non affidarsi alla sola UI

F-03 — SQL Injection nella ricerca prodotti

Stesso principio: dati non fidati non devono diventare sintassi

Pattern

Campo di ricerca
apparentemente innocuo.
Input controllato dall'utente
entra nella query SQL.
Una modifica minima del
parametro q cambia la
semantica della query.

Ciò che è stato dimostrato

Query manipulation minima.
Result set alterato.
Failure di input handling.

Ciò che NON è stato dimostrato

Dump tabella utenti
Furto password
RCE
Compromissione completa
DB

La remediation vera è la parametrizzazione delle query, non il filtering cosmetico dell'input.

F-04 — Reflected XSS nel greeting

Untrusted input che attraversa il boundary del browser

Pattern

Un parametro viene riflesso nella risposta HTML.

Il browser interpreta markup o script controllato dall'utente.

La PoC minima è locale, innocua e controllata.

Ciò che dimostra

Esecuzione di JavaScript nell'origine della web app.
Failure di output encoding contestuale.

Ciò che NON dimostra

Session theft

Compromissione admin

Persistence

Callback esterni

XSS e SQLi sembrano diverse, ma il pattern concettuale è simile: dati non fidati trattati come codice o sintassi.

F-05 — Information Disclosure sullo staging

Spesso un finding “low” abilita il passo successivo

Risorse esposte

robots.txt
/backup/appsettings.old
assets/app.js

Che cosa rivela

Environment di staging
API base path
Indicazioni su /admin/stats
Metadati applicativi non necessari

Perché è importante

Accelera la discovery
Riduce il costo
dell'esplorazione
Si correla con F-02

Conoscere un endpoint non dovrebbe bastare per comprometterlo. Il problema principale resta l'autorization mancante di F-02.

Il valore del reasoning: correlare i finding

Un report professionale non consegna solo schede isolate

Information Disclosure → endpoint admin scoperto → customer sessione valida → funzione admin accessibile

Customer legittimo → baseline su oggetto proprio → cambio dell'object ID → ordine di un altro customer

SQLi e XSS mostrano come input non fidato possa attraversare boundary diversi: database e browser.

Ma attenzione: "potrebbe succedere il passo successivo" ≠ "abbiamo dimostrato il passo successivo".

Il penetration test finisce nel report, non nel tool

Evidenza, impatto, remediation, retest

Evidence

Request/response, baseline, osservato vs atteso.

Impact

Che capability ottiene davvero l'attaccante?

Not demonstrated

Che cosa non possiamo affermare?

Remediation

Correzione immediata e strutturale.

Retest

Come dimostriamo che la fix funziona?

Oltre il digitale: fisico, radio, persone, processi

Il mezzo cambia. Il modello mentale resta identico.

Lo stesso pattern vale anche fuori dal browser

Scope → osservazione → test controllato → capability gained → next surface



- Nel mondo fisico e umano si ragiona sugli stessi concetti: confine, verifica, fallimento del controllo, impatto.
- Cambiano gli oggetti: badge, reception, Wi-Fi, porte, stampanti, USB, processi di help desk, visitatori, fornitori.

Esempio fisico 1 — La chiavetta USB “innocua”

Un test non distruttivo può già dire molto

Osservazione / ipotesi

Un supporto rimovibile sconosciuto viene trovato in azienda.

Qualcuno lo raccoglie? lo consegna all'IT? lo collega a un PC?

Capability dimostrata

Un oggetto non fidato è stato ritenuto abbastanza fidato da essere inserito in una workstation.

Remediation

Awareness

Procedure per supporti trovati

Device control / endpoint policy

Cultura del “non collegare ciò che non conosci”

Esempio fisico 2 — Il Wi-Fi che “esce” dall’edificio

Il perimetro fisico non coincide con il perimetro radio

Che cosa si verifica

Il segnale è misurabile e utilizzabile da fuori perimetro?

L'SSID che doveva servire solo l'area interna è raggiungibile da parcheggio o edificio adiacente?

Che tipo di autenticazione e segmentazione esistono?

Punto didattico

Il finding iniziale può essere soltanto RF exposure.

Il valore del finding dipende dal contesto: autenticazione, VLAN, reti ospiti, client isolation, servizi raggiungibili.

Anche qui: il test serio non parte dal crack, parte dalla comprensione del sistema.

Esempio fisico 3 — Entrare come personale delle pulizie / fornitore

Il vero target sono i controlli di accesso e di fiducia

Domande giuste

Chi controlla il badge?
Chi verifica l'identità?
Serve una escort?
Le zone operative sono segregate?
Le porte secondarie sono realmente controllate?

Capability

Accesso fisico a una zona che si riteneva protetta.

Nuova attack surface

Stampanti
Porte di rete
Documenti
Monitor
Sale riunioni
Asset labels

Oggetti “minori” che diventano interessanti: le stampanti

La superficie cambia quando cambia la capability

- Documenti lasciati nel vassoio di stampa.
- Display con informazioni o nomi di rete.
- Interfaccia web amministrativa o protocolli di stampa aperti.
- Rubriche, scan-to-mail, hostname, indirizzi IP, asset tag.
- Presenza su una rete che può diventare un punto di pivot o di raccolta informazioni.

Punto chiave

Una stampante non è interessante “sempre”.

Diventa interessante quando una nuova capability — accesso fisico o logico — la rende raggiungibile.

È lo stesso ragionamento usato per gli endpoint web del laboratorio.

Anche persone e processi fanno parte della superficie di attacco

Non esiste il "Nessus delle persone"

- Come vengono verificati visitatori, fornitori e pulizie?
- Come funziona l'help desk e chi può autorizzare un reset password?
- Come si gestiscono documenti, badge, sale riunioni, monitor e aree condivise?
- Quali processi fanno affidamento su fiducia implicita invece che su verifica?

Il pentester non testa solo tecnologie: testa se le assunzioni organizzative reggono nella pratica.

Perché i tool da soli non bastano

Il miglior investimento non è imparare più exploit, ma capire meglio i sistemi.

Il problema del pulsante "Scan"

Uno scanner produce output, non verità

Approccio debole

Metto IP

Clicco Scan

Aspetto

Leggo 137 findings

Esporto PDF

Approccio professionale

Leggo il sistema

Capisco i boundary

Formulo ipotesi

Uso il tool come supporto

Valido manualmente

Traduco in rischio e remediation

Che cosa il tool vede, e che cosa deve vedere il tester

La business rule non esce da sola da uno scanner

Il tool può vedere

- Porta 443 aperta
- Nginx presente
- Endpoint /api/orders/{id}
- Parametro q
- Cookie di sessione
- Header o banner

Il tester deve capire

- 1001 appartiene ad Alice e 1002 appartiene a Bob
- Alice non deve leggere l'ordine di Bob
- Admin stats richiede un ruolo specifico
- Il parametro q non deve cambiare la query
- Il parametro name non deve diventare markup attivo

Per diventare bravi pentester bisogna studiare i sistemi

Più capisci il sistema, più riconosci ciò che non dovrebbe essere possibile

Web / API

HTTP, sessioni, cookie, OAuth, ruoli, output encoding.

Database

SQL, query, privilegi, prepared statements.

Identity

AD, IAM, ruoli, gruppi, federazione, trust.

Network / Wireless

Routing, segmentazione, VLAN, 802.11, esposizione radio.

Cloud / Platform

Storage, IAM, networking, container, orchestrazione.

Quattro livelli diversi

Obiettivo del corso: portarvi oltre il semplice “tool user”

Tool user

Sa lanciare uno scanner o un tool.

Analyst

Sa leggere, filtrare e contestualizzare l'output.

Pentester

Sa formulare ipotesi, testarle e produrre evidenza.

Security expert

Comprende profondamente il sistema e individua failure che i tool non cercavano esplicitamente.

Il modello finale da portare via

Non partire dal tool: partire dal sistema

COME FUNZIONA?
COSA DOVREBBE PROTEGGERE?
DOVE SONO I TRUST BOUNDARY?
QUALI ASSUNZIONI STA FACENDO?
COSA SUCCEDDE SE NE ROMPO UNA?
COME POSSO TESTARLA?
QUALE TOOL MI AIUTA?
QUAL È L'EVIDENZA?

Messaggio finale del corso

La frase che vogliamo lasciare ai ragazzi

Il penetration testing non consiste nell'“hackerare un computer”. Consiste nel verificare, con autorizzazione ed evidenza, se i confini di sicurezza di un'organizzazione esistono davvero — e un confine può essere una API, un ruolo, una rete Wi-Fi, una reception, una procedura, una stampante o una decisione umana.

Un tool conosce le firme. Un pentester deve conoscere il sistema.